

Exact Functional Decomposition over Algebraic Number Fields

A factorization-free theory, exact certificates,
and a Wolfram Language implementation

Version 1.0.0 • 7 September 2026

Abstract

A polynomial whose coefficients contain radicals or algebraic `Root` objects does not require a special factorization theory in order to be functionally decomposed. In characteristic zero, each proposed degree of a normalized right component determines that component uniquely from the leading coefficients of the input. A finite formal-root recurrence constructs it using only arithmetic in the coefficient field. Repeated monic division then decides whether the input belongs to the polynomial subalgebra generated by the candidate.

We prove necessity, sufficiency, uniqueness, descent through arbitrary field extensions, completeness of recursive decomposition, and finite enumeration of all complete chains modulo affine changes. We give independently checkable positive and negative certificates, analyze arithmetic complexity and representation costs, and audit the derivative-factorization code in the motivating question. The accompanying Wolfram Language package implements the algorithms for explicit exact algebraic numbers, including complex ones. Worked examples cover the original radical polynomial, its degree-24 nested variant, a quintic `Root` coefficient, and nonunique decompositions.

Validation status. The mathematics is proved in this report. An independent exact number-field implementation completed 974 polynomial cases and 2,001 fixed-degree tests. The native Wolfram package and its 60-test regression suite were <i>not executed</i>: the available Wolfram service returned HTTP 404 and no local Wolfram kernel was present. Source review and independent mathematical checks are not substitutes for native execution.

Contents

1	Problem, scope, and main conclusion	2
2	Normalization and the coefficient field	3
3	The unique right-component candidate	3
3.1	A quadratic-cost formal-root recurrence	4
3.2	The Laurent-series interpretation	4
3.3	Why characteristic zero is explicitly required	5
4	Exact membership in the subalgebra $K[h]$	5
5	Descent, functoriality, and decomposition loci	6
5.1	A coefficient-space interpretation	6
6	One complete chain and all complete chains	7
6.1	Deterministic single-chain algorithm	7
6.2	Enumeration without repeated reassociations	8
7	Nonuniqueness and the Ritt perspective	8
7.1	Powers	8
7.2	A Chebyshev collision in the package's normalization	9
7.3	Intermediate fields as a structural interpretation	9
8	Exact positive and negative certificates	9
9	Worked examples	10
9.1	The original radical polynomial	10
9.2	The nested degree-24 example	11
9.3	An algebraic Root coefficient	11
9.4	Complex coefficients and affine normalization	12
10	Wolfram Language implementation and API	12
10.1	Loading and basic operation	12
10.2	Coefficient arithmetic and representation	12
10.3	Public functions	13
10.4	Checking a complete chain with public operations	14
10.5	Algorithmic organization	14
11	Complexity, coefficient growth, and practical limits	15
11.1	Field-operation bounds	15
11.2	Why algebraic arithmetic is not constant-cost	15
11.3	Storage and expression growth	15
12	Audit of the derivative-factorization approach	16
13	Verification record and reproducibility	17
13.1	What was actually executed	17
13.2	The differential baseline was not treated as an oracle	17
13.3	What has not been executed	18
13.4	Reproducing both validation layers	18
14	Conclusions and further implementation work	18
A	Complete Wolfram Language package source	20

1 Problem, scope, and main conclusion

The motivating question asks whether `Decompose` can be made to work on polynomials with radical coefficients, and presents a derivative-factorization implementation as an alternative [1]. The central distinction is between *functional decomposition*

$$p(x) = f(h(x)), \quad \deg f > 1, \quad \deg h > 1, \quad (1.1)$$

and ordinary multiplicative factorization. They are different problems. Irreducibility under multiplication neither implies nor is implied by indecomposability under composition.

For instance, $x^4 + 1 = (t^2 + 1) \circ x^2$ is functionally decomposable even though it is irreducible over $\mathbb{Q}$. Conversely, $x^4 + x$ is multiplicatively reducible but functionally indecomposable in characteristic zero, as the algorithm below immediately proves. An `Extension` option appropriate for multiplicative factorization is therefore not the fundamental missing ingredient.

Let

$$p(x) = \sum_{i=0}^n a_i x^i, \quad a_n \neq 0, \quad K = \mathbb{Q}(a_0, \dots, a_n) \subseteq \overline{\mathbb{Q}}. \quad (1.2)$$

Because there are finitely many algebraic coefficients, $K/\mathbb{Q}$ is a finite extension. The input expressions choose particular algebraic numbers, not unspecified conjugacy classes. A radical expression, an algebraic `Root`, and an `AlgebraicNumber` can denote the same element of K . Their outward forms should not determine the answer.

The central result. For every proper divisor d of n , there is at most one pair (f, h) satisfying (1.1) with $\deg h = d$, h monic, and $h(0) = 0$. Its only possible h is computed from the leading d coefficients of p by rational operations. A subsequent exact subalgebra-membership test either constructs f or certifies its nonexistence. No polynomial factorization, solving of coefficient equations, extraction of new algebraic numbers, or search through field extensions is needed.

This is a characteristic-zero realization of the classical factorization-free approach to polynomial decomposition, particularly the work of Kozen and Landau [2]. Faster and more general algorithmic treatments exist, including the tame-case work of von zur Gathen [3]. The elementary algorithm here is chosen for transparent proofs, straightforward exact implementation, and verifiable failure certificates, not as a claim to a new asymptotically optimal decomposition algorithm.

What is and is not claimed about the built-in function. The source discussion is from 2019. The currently retrieved Wolfram documentation illustrates symbolic and complex coefficients; it does not state an integer-only mathematical domain for `Decompose` [6]. The historical behavior reported in the question must not be promoted to a verified claim about every current Wolfram version. We could not run a current kernel in this environment. The package is an independent implementation; it does not modify `System`Decompose` or depend on its behavior.

Scope of the delivered code. The theory applies to any characteristic-zero field with exact arithmetic. The software intentionally accepts the narrower, decidable input class of explicit exact algebraic coefficients recognizable by Wolfram's algebraic number machinery. It does not accept approximate coefficients, symbolic parameters, transcendental coefficients, rational functions in x , or finite-characteristic arithmetic. These exclusions are input contracts, not assertions that broader mathematical variants are impossible.

2 Normalization and the coefficient field

Definition 2.1. A nonconstant polynomial $h \in K[x]$ is *normalized* if it is monic and $h(0) = 0$. A decomposition is *nontrivial* if both component degrees exceed one. A *complete chain* is a composition of polynomials of degree at least two, each of which is functionally indecomposable.

Suppose $p = f \circ g$ and write $b = \text{lc}(g) \neq 0$, $c = g(0)$. Set

$$h(x) = \frac{g(x) - c}{b}, \quad F(t) = f(bt + c). \quad (2.1)$$

Then $p = F \circ h$, and h is normalized. This replaces the arbitrary affine freedom at a composition interface by a canonical representative.

More explicitly, if $\ell(t) = bt + c$, the pairs (f, g) and $(f \circ \ell^{-1}, \ell \circ g)$ describe the same composition. Without removing this freedom there are generally infinitely many pairs even for a fixed degree pattern. We enumerate representatives modulo precisely these affine interface changes; we do not enumerate arbitrary choices of linear factors.

In a normalized complete chain

$$p = f_1 \circ f_2 \circ \cdots \circ f_r, \quad (2.2)$$

every component except f_1 is monic with constant term zero. There is no normalization requirement on f_1 : it must carry the input's leading coefficient and constant term. This convention also explains why a correct answer may look different from the radical expression in the question.

Remark 2.2. An algebraic number's selected embedding matters. For example, replacing $\sqrt{2}$ by $-\sqrt{2}$ changes the input polynomial in general. What is representation-independent is replacing an expression by another exact expression for the *same* algebraic number. No **PowerExpand**-style branch assumptions are legitimate here.

3 The unique right-component candidate

Fix $1 < d < n$ with $d \mid n$ and set $m = n/d$. Seek a normalized component

$$h(x) = x^d + b_1 x^{d-1} + \cdots + b_{d-1} x. \quad (3.1)$$

If $p = f(h)$, monicity of h forces $\text{lc}(f) = a_n$. All terms of $f(h) - a_n h^m$ have degree at most $(m-1)d = n-d$. Consequently,

$$[x^{n-k}]h(x)^m = \frac{a_{n-k}}{a_n}, \quad 1 \leq k \leq d-1. \quad (3.2)$$

These are only $d-1$ equations, exactly as many as the unknown coefficients of the normalized h .

Lemma 3.1 (Triangular recovery). *Over a characteristic-zero field, equations (3.2) have a unique solution $b_1, \dots, b_{d-1} \in K$. They can be solved successively by*

$$b_k = \frac{1}{m} \left(\frac{a_{n-k}}{a_n} - [x^{n-k}] \left(x^d + \sum_{j=1}^{k-1} b_j x^{d-j} \right)^m \right). \quad (3.3)$$

Proof. View a choice of $b_j x^{d-j}$ from one of the m factors as a degree deficit of j . A contribution of total deficit k either chooses b_k once and leading terms in every other factor, or uses only deficits smaller than k . The coefficient in the first case is mb_k . No b_j with $j > k$ can contribute. Thus the coefficient equation is linear in the new b_k , with nonzero coefficient m . Induction proves existence and uniqueness. Every step uses arithmetic in K alone. $\square$

Corollary 3.2 (Uniqueness for a fixed degree). *For fixed d , at most one normalized right component can occur. Once it occurs, its outer component is also unique.*

Proof. The first assertion follows from Lemma 3.1. Substitution $K[t] \rightarrow K[x]$, $f \mapsto f(h)$, is injective for nonconstant h , since a nonzero polynomial of degree r maps to one of degree rd . $\square$

3.1 A quadratic-cost formal-root recurrence

Repeatedly expanding powers in (3.3) is conceptually simple but unnecessary. Put

$$S(t) = 1 + \sum_{i=1}^{d-1} s_i t^i \pmod{t^d}, \quad s_i = \frac{a_{n-i}}{a_n}. \quad (3.4)$$

There is a unique $U(t) = 1 + \sum_{k=1}^{d-1} u_k t^k \pmod{t^d}$ with $U(t)^m = S(t) \pmod{t^d}$. It is the finite formal series with constant term one, customarily written $S(t)^{1/m}$. It is not a numerical choice of a complex m th-root branch. Its coefficients are in K .

Differentiating $U^m = S$ in formal power series gives $mSU' = US'$. Comparing coefficients of t^{k-1} yields

$$u_0 = 1, \quad u_k = \frac{1}{mk} \sum_{i=1}^k ((m+1)i - mk) s_i u_{k-i}, \quad 1 \leq k < d. \quad (3.5)$$

For completeness, the left side before isolating u_k is $mk u_k + m \sum_{i=1}^{k-1} (k-i) s_i u_{k-i}$; the right side is $\sum_{i=1}^k i s_i u_{k-i}$. Their difference is precisely (3.5). The candidate is

$$h_d(x) = x^d + u_1 x^{d-1} + \cdots + u_{d-1} x. \quad (3.6)$$

The first three coefficients illustrate the rational nature of the construction:

$$\begin{aligned} u_1 &= \frac{s_1}{m}, \\ u_2 &= \frac{s_2}{m} - \frac{m-1}{2m^2} s_1^2, \\ u_3 &= \frac{s_3}{m} - \frac{m-1}{m^2} s_1 s_2 + \frac{(m-1)(2m-1)}{6m^3} s_1^3. \end{aligned} \quad (3.7)$$

Only terms with indices $k < d$ are needed. The inner constant term is fixed at zero rather than inferred from a formal-root coefficient of order d .

3.2 The Laurent-series interpretation

At infinity, write

$$(p(x)/a_n)^{1/m} = x^d \left(1 + \frac{a_{n-1}}{a_n} x^{-1} + \cdots \right)^{1/m},$$

using the branch whose leading coefficient is one. The positive-degree polynomial part is exactly h_d . If $p = f(h)$ with $f(t) = a_n t^m + c_{m-1} t^{m-1} + \cdots$, then

$$(p/a_n)^{1/m} = h + \frac{c_{m-1}}{ma_n} + O(h^{-1}).$$

Thus the full polynomial part may contain a nonzero constant, but subtracting that constant gives the normalized h . This provides an approximate-root interpretation of the same exact finite computation. It does *not* justify accepting the candidate without testing the lower coefficients of p .

3.3 Why characteristic zero is explicitly required

The triangular construction divides by m ; the efficient recurrence also divides by k . Characteristic zero makes both safe. In characteristic two,

$$x^4 + x^2 = (t^2 + t) \circ x^2 = t^2 \circ (x^2 + x),$$

and the two right components are distinct normalized polynomials of the same degree. Therefore the fixed-degree uniqueness theorem itself can fail in wild characteristic. Even when m is invertible in positive characteristic, the particular recurrence (3.5) cannot simply be used if some $k < d$ is not invertible. The delivered code makes no finite-field claim.

4 Exact membership in the subalgebra $K[h]$

The candidate equations concern only the top of p . To test the whole polynomial, regard h as a polynomial “base.” No unknown outer coefficients or general-purpose solver are required.

Theorem 4.1 (Polynomial-base expansion). *Let $h \in K[x]$ be monic of degree $d \geq 1$. Every $p \in K[x]$ has a unique finite expansion*

$$p(x) = \sum_{j \geq 0} r_j(x) h(x)^j, \quad \deg r_j < d. \quad (4.1)$$

Moreover, $p \in K[h]$ if and only if every r_j is constant.

Proof. Repeated Euclidean division by the monic h gives $q_j = h q_{j+1} + r_j$, starting at $q_0 = p$. Quotient degree drops by d whenever it is at least d , so the process terminates. Substitution gives (4.1).

For uniqueness, suppose a nonzero finite combination $\sum r_j h^j$ vanishes, and let J be the largest index with $r_J \neq 0$. Its leading degree lies in the interval $[Jd, (J+1)d - 1]$, whereas every smaller index contributes a degree below Jd . Cancellation of its leading term is impossible. Hence all digits in a zero combination are zero.

If the digits are constants, the expansion itself defines an outer polynomial in $K[t]$. Conversely, an expression $p = \sum c_j h^j$ with $c_j \in K$ is a valid digit expansion. Uniqueness forces $r_j = c_j$. $\square$

Equivalently, $K[x]$ is a free $K[h]$ -module with basis $1, x, \dots, x^{d-1}$. The nonconstant digit coefficients are the coordinates that prevent membership in its rank-one submodule $K[h] \cdot 1$.

For $\deg p = md$, the digit list is $r_0, \dots, r_m$ and $r_m = a_n$. The outer candidate used by the implementation is

$$F_d(t) = \sum_{j=0}^m r_j(0) t^j. \quad (4.2)$$

It is an actual outer component precisely when all the digits are constant. On rejection the package reports both the nonzero residual $p - F_d(h_d)$ and one nonconstant digit coefficient. The latter identifies the exact failure, not merely an unsuccessful solver run.

Theorem 4.2 (Necessary and sufficient fixed-degree algorithm). *Let K have characteristic zero and let d be a proper divisor of $n = \deg p$. Construct h_d by (3.5)–(3.6), and compute the digits of p in base h_d . There exists a decomposition with right degree d if and only if every digit is constant. On success, (F_d, h_d) is the unique normalized pair.*

Proof. Any normalized right component of degree d must equal h_d by Lemma 3.1. Its existence is therefore equivalent to $p \in K[h_d]$, which Theorem 4.1 decides exactly. The outer component on success is (4.2); its degree is m because $r_m = a_n \neq 0$. Corollary 3.2 supplies uniqueness. $\square$

A small negative example. For $p = x^6 + x$, both proper right degrees are 2 and 3. In either case the leading coefficients force $h_d = x^d$. The digit $r_0 = x$ is nonconstant, so both degrees are impossible. This proves functional indecomposability even though six is composite. The proof works over every characteristic-zero extension of the coefficient field.

5 Descent, functoriality, and decomposition loci

Theorem 5.1 (Descent through field extensions). *Let L/K be any extension of characteristic-zero fields and $p \in K[x]$. For any $m, d > 1$, a decomposition of degree pattern (m, d) exists over L if and only if it exists over K . Every normalized pair over L already has both components in $K[x]$.*

Proof. The forward implication is the only one to prove. Normalize a pair over L by (2.1). Its right component is the unique solution of the triangular coefficient system. Formula (3.3) shows inductively that all its coefficients lie in K . Monic division of $p \in K[x]$ by this $h \in K[x]$ then produces digits in $K[x]$. Since they are constant over L , they are elements of K , and these are the coefficients of the outer component. The argument neither assumes that L/K is algebraic nor uses a Galois closure. $\square$

The descent mechanism is a standard consequence of the leading-coefficient method; related formulations appear explicitly in Wyman and Zieve [4]. Its relevance here is particularly strong:

Corollary 5.2. *For the algebraic input field K in (1.2), decomposition over K and over $\overline{\mathbb{Q}}$ gives the same normalized pairs and chains. Enlarging **Extension** cannot create a missing normalized decomposition.*

This conclusion is special to functional decomposition in characteristic zero. It contrasts with ordinary factorization, where $x^2 - 2$ is irreducible over $\mathbb{Q}$ but splits after adjoining $\sqrt{2}$. It also removes any need to decide whether a coefficient's minimal polynomial is solvable by radicals. A quintic Root is an entirely ordinary exact coefficient for this algorithm.

Proposition 5.3 (Compatibility with embeddings). *If $\sigma : K \hookrightarrow L$ is a field embedding, then candidate construction, digit expansion, and normalized decomposition commute with σ . In particular, conjugate input polynomials have the same accepted right degrees and the same complete degree patterns.*

Proof. Every construction uses finite additions, multiplications, and divisions by nonzero leading coefficients or nonzero integers. These operations commute with a field embedding. Nonzero coefficients stay nonzero, so the digit obstructions and acceptance decisions agree. $\square$

5.1 A coefficient-space interpretation

The finite tests also describe exactly where decompositions occur in families. Fix $n = md$ and regard $a_0, \dots, a_n$ as coordinates, with a_n invertible. The coefficients of h_d and all its base digits are regular functions in

$$K[a_0, \dots, a_n, a_n^{-1}]$$

when the base field has characteristic zero. In other words, their only variable denominators are powers of the leading coefficient; integer constants in denominators are harmless.

Proposition 5.4 (Fixed-degree decomposition locus). *Within the space of degree- n polynomials, the locus admitting right degree d is the common zero set of the nonconstant digit coefficients. It*

is isomorphic to the parameter space of arbitrary degree- m outer polynomials and normalized degree- d inner polynomials. Its dimension is $m + d$ and its codimension in the $(n + 1)$ -dimensional input coefficient space is $(m - 1)(d - 1)$.

Proof. Theorem 4.2 gives precisely the stated equations. The composition map from parameters (f, h) is polynomial. It is injective by normalized uniqueness, and its inverse on the locus is the regular candidate-and-digit construction just described. There are $m + 1$ outer coefficients, with the leading one nonzero, and $d - 1$ free inner coefficients. Hence the dimension is $(m + 1) + (d - 1) = m + d$. Subtracting from $n + 1 = md + 1$ gives $md + 1 - m - d = (m - 1)(d - 1)$. $\square$

This description is not an extra symbolic-parameter feature in the package; it is a mathematical explanation of specialization. As long as degree is preserved, the formulas specialize correctly, but exceptional parameter values may cause all obstructions to vanish. For example, $x^6 + \lambda x$ is indecomposable for nonzero λ and decomposes when $\lambda = 0$. Therefore no numerical “generic behavior” test can replace the exact coefficient tests.

6 One complete chain and all complete chains

Let

$$D(p) = \{d : 1 < d < \deg p, d \mid \deg p, p \in K[h_d]\}.$$

Testing all proper divisors computes every normalized nontrivial pair. There are at most $\tau(n) - 2$ such pairs for degree $n \geq 2$, where $\tau(n)$ is the number of positive divisors of n . Testing only prime right degrees is not complete: $(x^4 + x)^2$ has the atomic degree-four right component $x^4 + x$ and has no right component of degree two. Likewise, there is no justification for restricting the search to $d \leq \sqrt{n}$.

Lemma 6.1 (Minimal right degree gives an atom). *If $D(p)$ is nonempty and d is its smallest member, then h_d is indecomposable.*

Proof. If $h_d = u \circ v$ with both degrees greater than one, then $p = (F_d \circ u) \circ v$. Normalizing v yields a right component of p of degree $1 < \deg v < d$, contradicting minimality. $\square$

6.1 Deterministic single-chain algorithm

Test the proper divisors in increasing order. At the first success, record h_d as the rightmost component and repeat on F_d . Stop when no proper degree succeeds. The outer degree decreases by at least a factor of two at each step. The resulting list is returned outermost first.

Theorem 6.2 (Completeness of the single-chain algorithm). *The algorithm terminates and returns a complete normalized chain for every input of degree at least two. If it returns a singleton, the input is indecomposable over every characteristic-zero extension of K .*

Proof. Termination follows from strictly decreasing positive degree. Each recorded right component is indecomposable by Lemma 6.1. The final outer component is indecomposable because every possible right degree has been rejected by Theorem 4.2. All recorded identities are exact, so substituting them reconstructs p . Descent gives the extension-field claim. $\square$

6.2 Enumeration without repeated reassociations

For all complete chains, it is tempting to split recursively on every pair and on both sides. That repeats the same chain through different binary parenthesizations. The package instead selects the last atom first:

$$\mathcal{C}(p) = \begin{cases} \{(p)\}, & D(p) = \emptyset, \\ \bigcup_{\substack{d \in D(p) \\ h_d \text{ indecomposable}}} \{(c_1, \dots, c_r, h_d) : (c_1, \dots, c_r) \in \mathcal{C}(F_d)\}, & D(p) \neq \emptyset. \end{cases} \quad (6.1)$$

The implementation memoizes pair searches and chain lists locally within one call; it does not leave an unbounded global cache.

Theorem 6.3 (Complete enumeration modulo affine interfaces). *Formula (6.1) returns exactly all normalized complete chains, with no duplicates caused by binary reassociation. In particular, a specified ordered list of component degrees admits at most one normalized chain.*

Proof. In any normalized complete chain, the final component is indecomposable and has some degree $d \in D(p)$. By fixed-degree uniqueness, it must equal h_d , and the rest of the chain composes to the unique F_d . Induct on degree for the outer chain. This proves both inclusion directions and uniqueness for a fixed ordered degree list. Different selected final degrees cannot produce the same chain, and within one final degree the induction hypothesis rules out duplicates. Strict degree reduction proves finiteness. $\square$

A complete chain has at most $\lfloor \log_2 n \rfloor$ components, but the number of different chains need not be small. Enumerating all of them is necessarily output-sensitive. The interface intentionally separates one chain, all pairs, and all chains so that a user need not request the largest output to solve the ordinary decomposition problem.

Constants and affine polynomials. For software convenience, degree-zero and degree-one inputs return the singleton list containing the input, and the all-chains function returns a list containing that singleton. These are conventions, not complete chains of nonlinear atoms in the definition above. The certificate metadata marks indecomposability as not applicable below degree two. The zero polynomial's reported degree is $-\infty$.

7 Nonuniqueness and the Ritt perspective

Fixed-degree uniqueness does not say that all decompositions have the same ordered degrees. Ritt's characteristic-zero theory describes how complete chains can differ. In particular, their lengths and multisets of component degrees agree, while their order can change. The discussion of Zieve and Müller provides a detailed treatment and modern perspective [5]. None of the proofs of the algorithms above depends on invoking Ritt's classification as a black box.

7.1 Powers

For x^{12} , the normalized complete chains are exactly

$$(x^3, x^2, x^2), \quad (x^2, x^3, x^2), \quad (x^2, x^2, x^3).$$

For x^{30} there are six chains, corresponding to permutations of degrees 2, 3, 5. Composing two powers multiplies their exponents:

$$x^r \circ x^s = x^{rs}, \quad \text{not } x^{r+s}. \quad (7.1)$$

Even the correct multiplication should not be used to compress a *complete* chain, because it merges two atoms into a decomposable factor.

7.2 A Chebyshev collision in the package's normalization

The identities $T_2 \circ T_3 = T_3 \circ T_2 = T_6$ yield two normalized pairs for

$$T_6(x) = 32x^6 - 48x^4 + 18x^2 - 1 :$$

$$T_6 = (32t^3 - 48t^2 + 18t - 1) \circ x^2,$$

$$T_6 = (32t^2 - 1) \circ \left(x^3 - \frac{3}{4}x\right).$$

Their right degrees are different, so there is no conflict with Corollary 3.2. These are elementary test cases for all-chain enumeration; an implementation that always returns one degree pattern has not enumerated every chain.

More general collisions are not merely commutations of powers. For example, whenever the expressions have the required nontrivial degrees,

$$x^m \circ (x^r R(x^m)) = (x^r R(x)^m) \circ x^m.$$

The package need not recognize such templates in advance: both degree tests succeed directly.

7.3 Intermediate fields as a structural interpretation

A pair $p = f(h)$ gives a tower

$$K(p) \subseteq K(h) \subseteq K(x), \quad [K(x) : K(h)] = d, \quad [K(h) : K(p)] = m.$$

The degree statements follow from the degree of the corresponding rational maps, or equivalently from the generic polynomial equations for x and h . The normalization chooses a polynomial generator for a relevant intermediate field up to affine equivalence. This connects the algorithm to the function field viewpoint in the literature [2, 5]. The actual implementation does not construct intermediate fields, Galois groups, or monodromy groups; those structures are unnecessary for this finite characteristic-zero search.

8 Exact positive and negative certificates

A composition identity is a useful positive check, but it does not establish that a failed search was exhaustive or that returned factors are atoms. The package therefore exposes the evidence behind every proposed right degree.

A fixed-degree record contains d, m , the forced h_d , the digit list $r_0, \dots, r_m$, the constant-digit outer candidate F_d , a Boolean outcome, a possible obstruction, and the residual $p - F_d(h_d)$. An obstruction is a triple (j, i, c) with $i > 0$ and

$$c = [x^i]r_j \neq 0.$$

Digit and power indices in the software record are *zero-based* mathematical indices, although Wolfram list positions are one-based. The implementation selects the first nonzero nonconstant coefficient by increasing digit index and then increasing power.

Proposition 8.1 (A certificate checker independent of the search). *A fixed-degree record is sufficient to certify the outcome if a checker verifies all of the following: d is an admissible divisor; h_d is monic of degree d and has constant zero; the leading congruence (3.2) holds; every digit has degree less than d ; the reconstruction (4.1) equals p ; and the recorded status agrees with whether all digits are constant. On rejection, the claimed obstruction must be an actual nonzero nonconstant digit coefficient.*

Proof. The leading congruence and normalization identify the unique possible inner component by Lemma 3.1. Digit reconstruction and degree bounds identify the unique polynomial-base expansion by Theorem 4.1. Its constant or nonconstant nature decides existence. Thus neither the recurrence used to find h_d nor the long-division search must be trusted as part of this checker. $\square$

The delivered verifier follows this strategy: it raises h_d to m by polynomial multiplication and reconstructs the digit expression by Horner composition. It does not call the candidate recurrence, the monic-division routine, or the pair search. It additionally checks that the reported outer candidate is exactly the list of digit constants and that the residual is correct.

An exhaustive record includes a record for *every* proper divisor in increasing order. The verifier checks this entire list, including the absence of omissions and duplicates, and checks the accepted-degree summary. If all tests are negative, it certifies indecomposability. For prime degree the empty divisor list already suffices. To certify a complete returned chain, verify its composition identity and exhaustively certify every nonlinear component as indecomposable. All this can be done through the public API.

Trust boundary. These are mathematical certificates checked by exact computer algebra, not formally verified proof objects in a proof assistant. The search and checker still share scalar algebraic arithmetic and some basic coefficient-vector operations. They therefore do not provide implementation independence against every possible shared bug. The separate Python verifier uses a different exact number-field representation and an independent finite-binomial construction as further evidence. Native Wolfram execution remains a separate, explicitly unperformed validation step in this delivery.

9 Worked examples

9.1 The original radical polynomial

Put $s = \sqrt{2}$ and take the polynomial in the question:

$$\begin{aligned} p(x) = & 3 + 3s + (14 + 4s)x + (12 + 26s)x^2 \\ & + (56 + 8s)x^3 + (8 + 48s)x^4 + 48x^5 + 16sx^6. \end{aligned} \quad (9.1)$$

For $d = 2$, $m = 3$ and $s_1 = 48/(16s) = 3/s$. Hence $u_1 = 1/s$ and

$$h_2 = x^2 + \frac{x}{s}. \quad (9.2)$$

All base- h_2 digits are constant, giving

$$F_2(t) = 3 + 3s + (8 + 14s)t + (8 + 24s)t^2 + 16st^3. \quad (9.3)$$

Direct substitution verifies $p = F_2(h_2)$. This differs from the question's pair only by the affine normalization $g = sh_2$:

$$g = x + sx^2, \quad f(t) = 3 + 3s + (14 + 4s)t + (4 + 12s)t^2 + 8t^3,$$

and indeed $F_2(t) = f(st)$.

The alternative right degree $d = 3$ produces

$$h_3 = x^3 + \frac{3s}{4}x^2 + \left(\frac{15}{16} + \frac{s}{8}\right)x.$$

Its digit expansion is $p = r_0 + r_1h_3 + r_2h_3^2$ with

$$\begin{aligned} r_0 &= 3 + 3s + \left(\frac{51}{16} + \frac{3s}{4}\right)x + \left(\frac{3}{2} + \frac{51s}{16}\right)x^2, \\ r_1 &= 11 + 2s, \quad r_2 = 16s. \end{aligned}$$

The nonconstant r_0 proves that this degree is impossible. Thus the original polynomial has exactly one normalized complete chain, of degrees $(3, 2)$.

```
p = 3 + 3 Sqrt[2] + (14 + 4 Sqrt[2]) x +
    (12 + 26 Sqrt[2]) x^2 + (56 + 8 Sqrt[2]) x^3 +
    (8 + 48 Sqrt[2]) x^4 + 48 x^5 + 16 Sqrt[2] x^6;
chain = AlgebraicDecompose[p, x];
VerifyAlgebraicDecomposition[p, chain, x]
AlgebraicRightDecompose[p, x, 3]
(* Mathematical expectations: True; Missing["NotDecomposable",3]. *)
```

The displayed formulas are exact mathematical results independently checked in the bundled verifier; they are not represented as a native Wolfram session transcript.

9.2 The nested degree-24 example

Let $p_2(x) = p(x^4 - x + 1)$ and set $v = x^4 - x$. Since

$$h_2(v + 1) = v^2 + \left(2 + \frac{1}{s}\right)v + 1 + \frac{1}{s},$$

a normalized chain is

$$\begin{aligned} H_3(x) &= x^4 - x, \\ H_2(x) &= x^2 + \left(2 + \frac{1}{s}\right)x, \\ H_1(x) &= 141 + 105s + (168 + 142s)x \\ &\quad + (56 + 72s)x^2 + 16sx^3. \end{aligned}$$

Thus $p_2 = H_1 \circ H_2 \circ H_3$. The quartic H_3 is indecomposable: its only proposed right degree is two, the leading equations force x^2 , and the term $-x$ obstructs membership. The quadratic and cubic are indecomposable by prime degree. The returned degree sequence is $(3, 2, 4)$. In particular, a component of composite degree can be an atom.

9.3 An algebraic Root coefficient

Let α be the particular real algebraic number selected in Wolfram notation by

```
a = Root[#^5 - # - 1 &, 1];
```

and take

$$q(x) = (x^3 + \alpha x)^2 + (1 + \alpha)(x^3 + \alpha x) + 2.$$

For right degree three the algorithm recovers exactly

$$(t^2 + (1 + \alpha)t + 2, \quad x^3 + \alpha x).$$

No radical formula for α is requested or constructed. The coefficient relation $\alpha^5 - \alpha - 1 = 0$ is available to exact algebraic reduction. The same construction works for a nonreal root of this polynomial and for polynomials with coefficients from several algebraic extensions at once.

A useful validation example adds $(\alpha^5 - \alpha - 1)x^{40}$ to x^4 . The true polynomial is still x^4 , so its degree must be recomputed after coefficient reduction. The package does this before selecting divisors; it does not trust the apparent degree of unreduced input syntax.

9.4 Complex coefficients and affine normalization

Over $\mathbb{Q}(i, \sqrt{2})$,

$$(1 + i)(x^3 + ix)^2 + \sqrt{2} = ((1 + i)t^2 + \sqrt{2}) \circ (x^3 + ix).$$

For a less normalized input, choose

$$f(t) = (1 + \sqrt{2})t^2 + (2 - \sqrt{3})t + 7, \quad g(x) = (2 + \sqrt{3})x^3 + \sqrt{2}x + \sqrt{3}.$$

The unique normalized degree-three pair is

$$\left(f((2 + \sqrt{3})t + \sqrt{3}), \quad x^3 + \frac{\sqrt{2}}{2 + \sqrt{3}}x \right).$$

These examples emphasize that the outer leading coefficient, the original inner leading coefficient, and the original inner constant term are all handled without ad hoc assumptions.

10 Wolfram Language implementation and API

10.1 Loading and basic operation

The archive contains a self-contained application directory. Without installing anything into a system directory, load its top-level file:

```
Get["/absolute/path/AlgebraicDecomposition/AlgebraicDecomposition.wl"]
Clear[x];
AlgebraicDecompose[(x^4 + x)^2, x]
(* Expected: {x^2, x^4 + x} *)
```

Alternatively, place the complete directory under `$UserBaseDirectory/Applications` and use

```
Needs["AlgebraicDecomposition"]
```

The algorithm has no optional dependencies and no runtime network access. The Python verifier is separate and is not required to load or use the Wolfram package. The source targets modern Wolfram Language kernels with associations, `Failure`, and `RootReduce`; no minimum-version compatibility claim has been validated by native execution.

10.2 Coefficient arithmetic and representation

The implementation uses ascending dense coefficient vectors. For example, $3 + 2x + x^3$ is represented internally by $\{3, 2, 0, 1\}$. The zero polynomial has the unique vector $\{0\}$. Trailing zero entries are removed only after exact scalar normalization.

The scalar normalization boundary is the private function `red`. Exact integers and rationals pass directly. Other coefficients are processed with `RootReduce`, then checked for exact numeric membership in `Algebraics`. Wolfram documents `RootReduce` as reducing combinations of exact algebraic quantities to a single algebraic-number representation, and `Algebraics` provides the corresponding exact domain membership operation [7, 8]. The algorithm applies these operations to coefficients, not to polynomials containing the free variable.

An approximate real anywhere in the evaluated coefficient expression is rejected. So are nonnumeric symbolic parameters and quantities not certified as algebraic. A failed exact reduction returns a `Failure`; it is never treated as a nonzero coefficient or as evidence of indecomposability. After normalization, equality to the exact integer zero is used for zero tests. There is no tolerance, numerical sampling, or `PossibleZeroQ` in the algorithm.

This contract covers algebraic `Root` objects and exact arithmetic with them, not arbitrary `Root` expressions encoding transcendental equations. It also does not promise to recognize every sophisticated expression that happens mathematically to evaluate to an algebraic number. An explicit algebraic representation can be supplied when the initial expression lies outside the recognized domain.

10.3 Public functions

All lists of factors are outermost first. Mathematical equality, rather than the printed choice between radicals and `Root` objects, is the output contract.

`AlgebraicDecompose[p,x]`. Returns one complete normalized chain. The smallest successful right degree is chosen at each step, giving deterministic degree choices. An input of degree at most one returns its singleton list.

`AlgebraicDecompositions[p,x]`. Returns all normalized complete chains, modulo affine changes at internal interfaces. It implements (6.1), not an enumeration of all raw affine variants or all incomplete chains. The output can be large.

`AlgebraicDecompositionPairs[p,x]`. Returns all normalized nontrivial pairs, ordered by increasing right degree. An empty list means no nontrivial pair exists for a valid input. This interface is often a better diagnostic than full chain enumeration.

`AlgebraicRightDecompose[p,x,d]`. Returns the unique normalized pair for a specified proper divisor d . A valid degree with no decomposition returns `Missing["NotDecomposable",d]`. An invalid degree, such as a nondivisor, one, or the whole input degree, returns `Failure`. This distinction separates a proved negative answer from an invalid request.

`AlgebraicDecompositionData[p,x,d]`. Returns the fixed-degree certificate record. Its keys are `"Type"`, `"RightDegree"`, `"OuterDegree"`, `"Inner"`, `"OuterCandidate"`, `"Digits"`, `"Decomposable"`, `"Obstruction"`, and `"Residual"`. The outer candidate is not a valid factor when the status is false. The obstruction is `None` on success, otherwise an association with `"DigitIndex"`, `"Power"`, and `"Coefficient"`.

`AlgebraicDecompositionData[p,x]`. Returns an exhaustive report with `"InputDegree"`, `"TestedRightDegrees"`, `"AcceptedRightDegrees"`, `"Indecomposable"`, and `"Tests"`, in

addition to its type marker. For degree below two, the indecomposability field is `Missing["NotApplicable", "DegreeBelowTwo"]`.

VerifyAlgebraicDecompositionData[p,data,x]. Checks a fixed-degree or exhaustive report using Proposition 8.1. Malformed or inconsistent data returns `False`; invalid polynomial input returns `Failure`. The checker is intended for mathematical data, not as a sandbox for evaluating untrusted Wolfram expressions.

Composition and identity verification. `ComposeDecomposition[parts,x]` composes a list exactly; the empty list represents the identity x . `VerifyAlgebraicDecomposition[p,parts,x]` checks only the composition identity. It does not assert completeness, normalization, or a particular degree pattern. For example, $\{x^4\}$ passes the identity check for $p = x^4$ but is not a complete nonlinear chain.

10.4 Checking a complete chain with public operations

The following pattern distinguishes the identity check from the atomicity checks:

```
chain = AlgebraicDecompose[p, x];
identityOK = VerifyAlgebraicDecomposition[p, chain, x];
reports = AlgebraicDecompositionData[#, x] & /@ chain;
certificatesOK = And @@ MapThread[
  VerifyAlgebraicDecompositionData[#1, #2, x] &, {chain, reports}];
atomsOK = AllTrue[reports, TrueQ[#["Indecomposable"]] &];
(* For degree(p)>=2, require identityOK && certificatesOK && atomsOK. *)
```

In production code, check for `Failure` before inspecting returned lists or associations. Inputs should use an unassigned symbol as the polynomial variable; ordinary Wolfram evaluation happens before the function patterns are matched.

10.5 Algorithmic organization

The fixed-degree engine consists of three small components: the formal-root recurrence, monic long division, and detection of nonconstant digits. Outer coefficients are taken directly from digit constants. Composition and certificate checking use explicit polynomial convolution and Horner schemes. Loop exits are explicit: the nested obstruction scan uses a locally tagged `Catch/Throw`, and pair search records its result before `Break`. Wolfram's `Return` inside a `Do` loop exits that loop, not necessarily the surrounding function, so it must not be used as an implicit multilevel exit [11]. There is no call to `Factor`, `Decompose`, `Solve`, or a radical denesting routine in the search. Complete source is included in Appendix A and in the loadable package file.

There are deliberately no speculative options for extensions, numerical tolerances, or heuristic fallback solvers. Unsupported input or a failed algebraic normalization does not silently change the mathematical problem. The implementation also does not automatically convert every output to radicals: such a conversion may be impossible or needlessly expensive, and would not improve correctness.

11 Complexity, coefficient growth, and practical limits

11.1 Field-operation bounds

For a fixed proposed right degree d , the recurrence performs $1 + 2 + \dots + (d - 1) = O(d^2)$ scalar operations. A dense division of a polynomial of degree N by a monic degree- d polynomial costs $O((N - d + 1)d)$ field operations. Applying this repeatedly at degrees $n, n - d, n - 2d, \dots$ costs

$$O\left(d \sum_{j=0}^{m-1} (n - jd)\right) = O(n^2), \quad m = n/d.$$

Thus the fixed-degree test is $O(n^2)$ in the dense field-operation model, with $O(n)$ field elements of working storage for its coefficient vectors and digits. This agrees with the classical quadratic fixed-degree approach [2]; it is not a bit-complexity bound.

Testing every proper divisor gives $O(\tau(n)n^2)$ field operations. Along one complete chain, subsequent outer degrees are divisors of n and at most half the previous degree. Since $\tau(n_j) \leq \tau(n)$,

$$\sum_j O(\tau(n_j)n_j^2) \leq O\left(\tau(n)n^2 \sum_{j \geq 0} 4^{-j}\right) = O(\tau(n)n^2).$$

Enumeration of all chains adds the cost of its search graph and its output; no polynomial bound in input size alone is claimed for that interface. Storing all degree certificates simultaneously also costs more than storing one fixed-degree test.

11.2 Why algebraic arithmetic is not constant-cost

Let $D = [K : \mathbb{Q}]$. In a common primitive-element representation $K = \mathbb{Q}(\theta)$, an element is a rational vector of length D , interpreted modulo the minimal polynomial of θ . Multiplication, inversion, and rational height growth all have costs depending on D and coefficient sizes. A concise expression with many independent radicals can generate a field whose degree is the product of their individual degrees. Therefore a polynomial field-operation count does not imply polynomial running time in the length of an arbitrary radical expression.

The delivered Wolfram backend favors a simple normalization boundary using `RootReduce`. It may repeatedly do expensive algebraic work, especially when many unrelated generators occur. A future optimized backend could convert all input coefficients once to a common field, operate on rational coordinate vectors, and convert only at the output boundary. `ToNumberField` is relevant to such a design [9]. That backend is *not* an undocumented option in this release. The independent Python verifier does use explicit exact number fields, but it is not a replacement runtime dependency for the Wolfram package.

11.3 Storage and expression growth

The implementation is dense. A sparse expression such as $x^{1000000} + x$ will create a very long vector even though its textual input is short. Repeated `RootReduce`, dense convolution, large exact coefficients, and all-chain enumeration are the principal practical limits. No benchmark claim for a particular Wolfram version is possible without native execution. The measured Python verification time is merely a reproducibility record, not a comparison with `Decompose` or a performance promise.

12 Audit of the derivative-factorization approach

The identity

$$p'(x) = f'(g(x))g'(x)$$

is mathematically useful. In characteristic zero, after suitable scalar normalization, the derivative of a right component divides p' . An exhaustive enumeration of derivative divisors, followed by integration, normalization, and exact membership testing, can therefore produce a correct algorithm. The factorization-free method replaces this potentially large candidate set by one forced candidate per right degree.

Several details in the posted code require correction independently of its choice of method.

One distinct derivative factor is not an obstruction. The early return when the non-constant factor list has length below two is incorrect. For $p = x^4$, $p' = 4x^3$ has a single distinct nonconstant factor x with multiplicity three, yet $p = x^2 \circ x^2$. Multiplicity cannot be ignored when deciding whether candidate derivatives exist.

Repeated-factor subsets create duplicates. Expanding multiplicities into repeated copies and taking all subsets can visit $2^{\sum e_i}$ subsets, with many yielding the same product. Divisors of $\prod q_i^{e_i}$ are more naturally indexed by multiplicity tuples $0 \leq j_i \leq e_i$, of which there are $\prod (e_i + 1)$. Even that is an avoidable combinatorial search for the present characteristic-zero task.

The candidate degree must divide the input degree. If a proposed derivative has degree $d - 1$, then d must be a proper divisor of n . The posted degree bound on derivative products does not enforce this. Using n/d subsequently as the number of outer coefficients can therefore produce a noninteger degree and an invalid coefficient system. Divisibility must be checked before constructing the outer polynomial.

Recurring only on the outer factor needs a justification. A successfully found inner factor need not be indecomposable. The provided candidate-subset order is not a proof that the first successful candidate has minimum degree. Therefore repeatedly decomposing only the first list element does not by itself certify that every final component is an atom. Our single-chain algorithm does recurse only on the outer component, but only after Lemma 6.1 has supplied the missing justification.

The final power rewrite changes the composition. The replacement of adjacent powers by x^{n+m} is algebraically wrong; (7.1) requires x^{nm} . For example, adjacent cubic powers compose to x^9 , not x^6 . Correcting addition to multiplication fixes the identity but still destroys completeness by merging nonlinear factors.

Exact checking must remain exact. A heuristic possible-zero predicate is not the same interface contract as coefficientwise exact algebraic reduction. Likewise, the existence of one solver result is not an exhaustive negative certificate when no result is returned. For explicit algebraic coefficients, reduction to canonical exact numbers and the finite obstruction test make the decision boundary explicit.

The historical comment in the source thread describes an old implementation context for `Decompose`; it does not establish that the derivative approach is the only known algorithm. The factorization-free leading-term approach substantially predates that discussion [2, 1].

13 Verification record and reproducibility

13.1 What was actually executed

The archive includes `Verification/verify_exact.py`, an independent Python implementation using SymPy 1.14.0 exact rational and algebraic-number fields. Polynomial arithmetic in the test engine is explicit vector arithmetic; it does not call SymPy’s decomposition routine to decide the expected result. Its candidate recurrence is also cross-checked against a separate truncated binomial expansion

$$(1 + w)^{1/m} \equiv \sum_{j=0}^{d-1} \binom{1/m}{j} w^j \pmod{t^d}, \quad w \in tK[t].$$

This second construction does not reuse the recurrence.

The successful run recorded in `Verification/report.json` has the following scope:

Executed independent check	Count
Polynomial cases	974
Fixed-degree candidate and digit tests	2,001
Complete chains checked	1,042
Known normalized pairs recovered	95
Rational differential comparisons with SymPy	848
Degree-pattern disagreements with that baseline	14

There are 14 fixed examples, 810 exhaustively enumerated monic rational polynomials of degrees four and six with lower coefficients in $\{-1, 0, 1\}$, and 30 generated cases in each of five fields:

$$\mathbb{Q}, \quad \mathbb{Q}(\sqrt{2}), \quad \mathbb{Q}(i), \quad \mathbb{Q}(\sqrt{2}, \sqrt{3}), \quad \mathbb{Q}(\alpha), \quad \alpha^5 - \alpha - 1 = 0.$$

The generator uses seed 206618. The generated cases include nonmonic outer and inner components and nonzero inner constants. Verification checks exact recomposition, candidate agreement, leading congruence, digit reconstruction, acceptance equivalence, normalization, atomicity, and uniqueness of enumerated chains. Ritt’s degree-multiset invariant is checked as an additional property. No approximate numerical equality is used.

13.2 The differential baseline was not treated as an oracle

SymPy’s documented `decompose` function provides a useful independent comparison [12], but in this run 14 rational cases had different degree patterns. These are explicitly reported, not counted as agreement. For example, the installed SymPy 1.14.0 call on

$$p(x) = -8x^6 + 24x^5 - 34x^4 + 38x^3 - 29x^2 + 14x - 7$$

with domain $\mathbb{Q}$ returned a singleton, whereas exact multiplication verifies

$$p = (-8t^2 + 14t - 7) \circ \left(x^3 - \frac{3}{2}x^2 + x \right).$$

The independent engine recovered the latter pair and checked it exactly. The report records the first three discrepancies for reproduction. Its **PASS** status refers to its own stated exact invariants, *not* to universal agreement with the baseline. This observation is version- and input-specific; it is not a claim about every SymPy release.

13.3 What has not been executed

The Wolfram connection was discovered and an evaluator call was attempted. Both its context and evaluation endpoints returned HTTP 404, and there was no local Wolfram kernel. The native package therefore has no recorded successful native load or native test run in this delivery. The 60 authored `VerificationTest` cases include the original example, mixed fields, real and nonreal quintic roots, `AlgebraicNumber`, algebraic cancellations, all-chain examples, malformed input, and deliberately corrupted certificates. One of these tests additionally wraps 24 seeded compositional property cases. These native tests are supplied for execution, not presented as already passed.

13.4 Reproducing both validation layers

With a Wolfram kernel available, run from the application directory:

```
wolframscript -file Tests/RunTests.wl
```

In a notebook, a report can instead be obtained directly:

```
TestReport["/absolute/path/AlgebraicDecomposition/Tests/AlgebraicDecomposition.wlt"]
```

The runner prints the kernel version and the test report and uses the available report success property; the current documented property is `"ReportSucceeded"` [10]. A fallback recognizes the older `"AllTestsSucceeded"` property when present. An unknown report interface is not silently counted as success.

For the independent check, use Python 3.10 or later and install the pinned verification dependency from the supplied requirements file. Then run:

```
python Verification/verify_exact.py --output Verification/report.json
```

The Python script is a mathematical verification companion, not a Wolfram syntax checker. The bundle also records a lightweight delimiter/string/comment scan of the Wolfram files; that scan is explicitly not a full Wolfram parser.

14 Conclusions and further implementation work

For explicit exact algebraic coefficients, functional decomposition has a clean finite solution: normalize the inner component, force its coefficients from the top of the input, and check the lower coefficients through polynomial base digits. Each divisor contributes exactly one candidate. Successful pairs descend to the original coefficient field, and rejected candidates carry exact algebraic obstructions. Complete chains and all normalized alternatives follow by elementary recursion.

The package realizes this theory without derivative factorization or coefficient solving. Its principal remaining validation task is native Wolfram execution, for which the full regression suite is included. Performance work would naturally focus on a common-number-field backend, sparse or fast polynomial arithmetic, and lazy enumeration of very large chain families. Those are improvements to engineering and scale, not missing cases in the characteristic-zero mathematical criterion.

References

- [1] V. Reshetnikov and contributors, *Is it possible to make Decompose work with coefficients containing radicals?* Mathematica Stack Exchange, question 206618 and answer 206622, 2019. Source question, contributed code, and historical discussion. <https://mathematica.stackexchange.com/questions/206618/>
- [2] D. Kozen and S. Landau, *Polynomial decomposition algorithms*, Journal of Symbolic Computation **7** (1989), 445–456. Author-hosted paper: <https://www.cs.cornell.edu/~kozen/Papers/poly.pdf>
- [3] J. von zur Gathen, *Functional decomposition of polynomials: the tame case*, Journal of Symbolic Computation **9** (1990), 281–299. Bibliographic details verified against the author’s publication list: <https://vonzurgathen.online/list-of-publications/>
- [4] B. K. Wyman and M. E. Zieve, *Two questions on polynomial decomposition*, arXiv:1301.5211, 2013. In particular, the coefficient-descent argument in Proposition 2.1. <https://arxiv.org/html/1301.5211v1>
- [5] M. E. Zieve and P. Müller, *On Ritt’s polynomial decomposition theorems*, arXiv:0807.3578, 2008. <https://arxiv.org/abs/0807.3578>
- [6] Wolfram Research, *Decompose*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Decompose.html>
- [7] Wolfram Research, *RootReduce*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>
- [8] Wolfram Research, *Algebraics*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/Algebraics.html>
- [9] Wolfram Research, *ToNumberField*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/ToNumberField.html>
- [10] Wolfram Research, *TestReportObject*, Wolfram Language documentation. <https://reference.wolfram.com/language/ref/TestReportObject.html>
- [11] Wolfram Research, *Return*, Wolfram Language documentation, especially the example under “Possible Issues.” <https://reference.wolfram.com/language/ref/Return.html>
- [12] SymPy development team, *Polynomial manipulation: reference*, SymPy documentation; local verification used version 1.14.0. <https://docs.sympy.org/latest/modules/polys/reference.html>

All online references were consulted on 7 September 2026. The mathematical arguments in this report are self-contained; bibliography entries distinguish algorithmic antecedents, structural background, and implementation contracts.

A Complete Wolfram Language package source

The following source is embedded in this L^AT_EX file so the article can be compiled on its own. The same source is supplied as `Kernel/AlgebraicDecomposition.wl`, which is the file to load rather than copying code out of a PDF. The top-level loader and `Kernel/init.m` are small path-relative wrappers. The examples, 60 native tests, and independent Python verifier are separate files in the archive.

```

1 (* ::Package:: *)
2 (* AlgebraicDecomposition 1.0.0 -- exact characteristic-zero functional
3    decomposition over algebraic numbers. See article/article.pdf.
4    No Factor, Decompose, Solve, approximate zero test, or radical denesting
5    is used by the decomposition algorithm. SPDX-License-Identifier: MIT *)
6
7 BeginPackage["AlgebraicDecomposition"];
8
9 AlgebraicDecompose::usage =
10 "AlgebraicDecompose[p,x] returns one complete composition chain, outermost first. It chooses the
11    smallest successful right degree at each step. All components except the first are monic with
12    zero constant term. Exact algebraic coefficients are required.";
13 AlgebraicDecompositions::usage =
14 "AlgebraicDecompositions[p,x] returns all complete normalized composition chains, modulo affine
15    changes at internal interfaces. Enumeration may have large output.";
16 AlgebraicDecompositionPairs::usage =
17 "AlgebraicDecompositionPairs[p,x] returns all normalized nontrivial pairs {f,g} with p=f(g(x)),
18    ordered by increasing degree of g.";
19 AlgebraicRightDecompose::usage =
20 "AlgebraicRightDecompose[p,x,d] returns the unique pair {f,g} with degree(g)=d, g monic and g(0)=0,
21    or Missing["NotDecomposable",d]. An invalid degree or input returns Failure.";
22 AlgebraicDecompositionData::usage =
23 "AlgebraicDecompositionData[p,x,d] gives an exact fixed-degree test, including the forced inner
24    candidate, its base-g digits and an obstruction. AlgebraicDecompositionData[p,x] gives tests for
25    every proper divisor of degree(p).";
26 VerifyAlgebraicDecompositionData::usage =
27 "VerifyAlgebraicDecompositionData[p,data,x] verifies fixed-degree or exhaustive data by leading-
28    coefficient congruence, digit reconstruction, and exact coefficient arithmetic; it does not rerun
29    the candidate recurrence or the division search. Returns False for malformed data and Failure
30    for invalid polynomial input.";
31 ComposeDecomposition::usage =
32 "ComposeDecomposition[parts,x] composes an outermost-first list of exact algebraic polynomials. The
33    empty list represents x.";
34 VerifyAlgebraicDecomposition::usage =
35 "VerifyAlgebraicDecomposition[p,parts,x] checks the exact polynomial identity p=ComposeDecomposition
36    [parts,x]. It checks identity, not completeness or normalization.";
37
38 Begin["Private"];
39
40 $failureTag = Unique["AlgebraicDecompositionFailure"];
41 fail[tag_String, message_String] :=
42   Throw[Failure[tag, <|"MessageTemplate" -> message|>], $failureTag];
43 invalid[] := Failure["InvalidArguments", <|"MessageTemplate" ->
44   "Use a documented argument sequence and an unassigned polynomial variable."|>];
45
46 (* This is the only algebraic-number normalization boundary. In particular,
47    RootReduce is applied to scalars, never to a polynomial containing x. *)
48 red[z_] := Module[{r},
49   If[MatchQ[z, _Integer | _Rational], Return[z]];
50   r = Quiet[Check[RootReduce[z], $Failed]];
51   If[r === $Failed || !FreeQ[r, _Real] ||
52     !TrueQ[NumericQ[r]] || !TrueQ[Element[r, Algebraics]],
53     fail["AlgebraicArithmetic", "Exact algebraic-number reduction failed."]];
54   r
55 ];
56
57 (* Ascending dense coefficient vectors. Zero is always {0}. *)
58 trim[v_List] := Module[{w = v},
59   If[w === {}, Return[{0}]];
60   While[Length[w] > 1 && Last[w] === 0, w = Most[w]];
61   w
62 ];
63 zeroQ[v_List] := v === {0};

```

```

52 vectorDegree[v_List] := If[zeroQ[v], -Infinity, Length[v] - 1];
53
54 prepare[p_, x_Symbol] := Module[{q, c},
55   q = Quiet[Check[Expand[p], $Failed]];
56   If[q === $Failed || !TrueQ[PolynomialQ[q, x]],
57     fail["NotPolynomial", "The input must be a univariate polynomial."];
58   c = CoefficientList[q, x];
59   If[c === {}, c = {0}];
60   If[!FreeQ[c, _Real],
61     fail["InexactCoefficient", "Approximate coefficients are not accepted."];
62   If[!(And @@ (TrueQ[NumericQ[#]] & /@ c)),
63     fail["NonAlgebraicCoefficient", "Every coefficient must be an explicit exact algebraic number;
64     symbolic parameters are not accepted."];
65   trim[red /@ c]
66 ];
67
68 expression[v_List, x_] := Expand[Fold[#1 x + #2 &, 0, Reverse[v]]];
69 properDegrees[n_Integer] := If[n < 4, {}, Select[Divisors[n], 1 < # < n &]];
70
71 add[a_List, b_List] := trim[red /@
72   (PadRight[a, Max[Length[a], Length[b]]] +
73    PadRight[b, Max[Length[a], Length[b]]]);
74 subtract[a_List, b_List] := add[a, -b];
75 multiply[a_List, b_List] := Module[{r, i, j},
76   If[zeroQ[a] || zeroQ[b], Return[{0}]];
77   r = ConstantArray[0, Length[a] + Length[b] - 1];
78   Do[If[a[[i]] != 0,
79     Do[If[b[[j]] != 0,
80       r[[i + j - 1]] = r[[i + j - 1]] + a[[i]] b[[j]],
81       {j, Length[b]}], {i, Length[a]}];
82   trim[red /@ r]
83 ];
84 compose[a_List, b_List] := Fold[add[multiply[#1, b], {#2}] &,
85   {0}, Reverse[a]];
86 digitCompose[digits_List, h_List] :=
87   Fold[add[multiply[#1, h], #2] &, {0}, Reverse[digits]];
88
89 (* Coefficients u_k of (1+s_1 t+...)^{(1/m)}, truncated before t^d.
90   m S U' = U S' gives the O(d^2) recurrence used here. *)
91 rightCandidate[c_List, d_Integer] := Module[{n, m, s, u, k, i},
92   n = Length[c] - 1; m = Quotient[n, d];
93   s = Table[red[c[[n - k + 1]]/Last[c]], {k, 0, d - 1}];
94   u = ConstantArray[0, d]; u[[1]] = 1;
95   Do[u[[k + 1]] = red[Total[Table[
96     (((1 + 1/m) i - k) s[[i + 1]] u[[k - i + 1]]),
97     {i, 1, k}]]/k], {k, 1, d - 1}];
98   Prepend[Reverse[u], 0]
99 ];
100
101 (* Exact monic long division. The top coefficient is canceled by assignment;
102   lower positions are reduced immediately, keeping zero recognition exact. *)
103 monicDivide[a_List, h_List] := Module[{r = a, q, d, n, k, j, t},
104   d = Length[h] - 1; n = Length[a] - 1;
105   If[n < d, Return[{0}, a]];
106   q = ConstantArray[0, n - d + 1];
107   Do[t = r[[k + d + 1]]; q[[k + 1]] = t;
108     If[t != 0,
109       Do[r[[k + j + 1]] = red[r[[k + j + 1]] - t h[[j + 1]],
110         {j, 0, d - 1}];
111       r[[k + d + 1]] = 0,
112       {k, n - d, 0, -1}];
113   {trim[q], trim[Take[r, d]]}
114 ];
115
116 baseDigits[c_List, h_List] := Module[{q = c, qr, out = {}},
117   While[!zeroQ[q],
118     qr = monicDivide[q, h];
119     AppendTo[out, qr[[2]]]; q = qr[[1]];
120   ];
121
122 (* Obstruction indices are mathematical, zero-based digit/power indices. *)
123 obstruction[digits_List] := Module[{j, k, tag},

```

```

124 (* Tagged Throw leaves both loops; Return would leave only the inner Do. *)
125 Catch[
126   Do[Do[If[digits[[j, k]] != 0,
127     Throw[<|"DigitIndex" -> j - 1, "Power" -> k - 1,
128       "Coefficient" -> digits[[j, k]]|>, tag]],
129     {k, 2, Length[digits[[j]]}], {j, Length[digits]}];
130   None, tag]
131 ];
132
133 (* Internal records keep vectors, not expressions in a public variable. *)
134 testDegree[c_List, d_Integer] := Module[{h, digits, obs},
135   h = rightCandidate[c, d]; digits = baseDigits[c, h];
136   obs = obstruction[digits];
137   <|"RightDegree" -> d, "OuterDegree" -> Quotient[Length[c] - 1, d],
138     "InnerVector" -> h, "OuterVector" -> (First /@ digits),
139     "DigitVectors" -> digits, "Decomposable" -> (obs === None),
140     "Obstruction" -> obs|>
141 ];
142
143 publicTest[c_List, t_Association, x_] := <|
144   "Type" -> "DegreeTest", "RightDegree" -> t["RightDegree"],
145   "OuterDegree" -> t["OuterDegree"],
146   "Inner" -> expression[t["InnerVector"], x],
147   "OuterCandidate" -> expression[t["OuterVector"], x],
148   "Digits" -> (expression[#, x] & /@ t["DigitVectors"]),
149   "Decomposable" -> t["Decomposable"], "Obstruction" -> t["Obstruction"],
150   "Residual" -> expression[subtract[c,
151     compose[t["OuterVector"], t["InnerVector"]]], x]|>;
152
153 checkDegree[c_List, d_] := If[!IntegerQ[d] || d < 2 ||
154   d >= Length[c] - 1 || Mod[Length[c] - 1, d] != 0,
155   fail["InvalidRightDegree", "The right degree must be a proper divisor d of the polynomial degree
156     with 1 < d < n."]];
157
158 firstPair[c_List] := Module[{t, d, result = None},
159   Do[t = testDegree[c, d];
160     If[TrueQ[t["Decomposable"]],
161       result = {t["OuterVector"], t["InnerVector"]}; Break[]];
162   {d, properDegrees[Length[c] - 1]};
163 ];
164
165 allPairs[c_List] := Module[{out = {}, t, d},
166   Do[t = testDegree[c, d];
167     If[TrueQ[t["Decomposable"]],
168       AppendTo[out, {t["OuterVector"], t["InnerVector"]}]];
169   {d, properDegrees[Length[c] - 1]};
170 ];
171
172 oneChain[c_List] := Module[{v = c, out = {}, pair},
173   pair = firstPair[v];
174   While[pair != None,
175     (* Minimal successful right degree implies an indecomposable inner factor. *)
176     PrependTo[out, pair[[2]]]; v = pair[[1]];
177     pair = firstPair[v];
178     Prepend[out, v]
179 ];
180
181 allChains[c_List] := Module[{pairs, walk},
182   pairs[v_List] := pairs[v] = allPairs[v];
183   walk[v_List] := walk[v] = Module[{ps, out = {}, pr, tails},
184     ps = pairs[v];
185     If[ps === {}, {v}],
186     Do[If[pairs[pr[[2]]] === {},
187       tails = walk[pr[[1]]];
188       out = Join[out, (Append[#, pr[[2]] & /@ tails)]]; {pr, ps}];
189     out];
190   walk[c]
191 ];
192
193 AlgebraicDecompose[p_, x_Symbol] := Catch[
194   expression[#, x] & /@ oneChain[prepare[p, x]], $failureTag];
195 AlgebraicDecompositions[p_, x_Symbol] := Catch[

```



```

196 Map[expression[#, x] &, allChains[prepare[p, x]], {2}], $failureTag];
197 AlgebraicDecompositionPairs[p_, x_Symbol] := Catch[
198   Map[expression[#, x] &, allPairs[prepare[p, x]], {2}], $failureTag];
199 AlgebraicRightDecompose[p_, x_Symbol, d_] := Catch[Module[{c, t},
200   c = prepare[p, x]; checkDegree[c, d]; t = testDegree[c, d];
201   If[TrueQ[t["Decomposable"]],
202     {expression[t["OuterVector"], x], expression[t["InnerVector"], x]},
203     Missing["NotDecomposable", d]], $failureTag];
204
205 AlgebraicDecompositionData[p_, x_Symbol, d_] := Catch[Module[{c},
206   c = prepare[p, x]; checkDegree[c, d]; publicTest[c, testDegree[c, d], x]],
207   $failureTag];
208 AlgebraicDecompositionData[p_, x_Symbol] := Catch[Module[{c, ds, ts, good},
209   c = prepare[p, x]; ds = properDegrees[Length[c] - 1];
210   ts = publicTest[c, testDegree[c, #], x] & /@ ds;
211   good = (#["RightDegree"] & /@ Select[ts, TrueQ[#["Decomposable"]]] &)];
212   <| "Type" -> "AllDegreeTests", "InputDegree" -> vectorDegree[c],
213   "TestedRightDegrees" -> ds, "AcceptedRightDegrees" -> good,
214   "Indecomposable" -> If[Length[c] < 3,
215     Missing["NotApplicable", "DegreeBelowTwo"], good === {}],
216   "Tests" -> ts|>, $failureTag];
217
218 ComposeDecomposition[parts_List, x_Symbol] := Catch[Module[{vs},
219   vs = prepare[#, x] & /@ parts;
220   expression[Fold[compose[#2, #1] &, {0, 1}, Reverse[vs]], x]], $failureTag];
221 VerifyAlgebraicDecomposition[p_, parts_List, x_Symbol] := Catch[
222   Module[{c, vs, v}, c = prepare[p, x]; vs = prepare[#, x] & /@ parts;
223   v = Fold[compose[#2, #1] &, {0, 1}, Reverse[vs]];
224   zeroQ[subtract[c, v]], $failureTag];
225
226 (* A deliberately different checker: no rightCandidate, monicDivide,
227   baseDigits, firstPair, or allPairs call occurs below. *)
228 verifyTest[c_List, t_, x_] := Module[
229   {keys, d, n, m, h, f, digits, obs, hm, k, res},
230   If[!AssociationQ[t], Return[False]];
231   keys = {"Type", "RightDegree", "OuterDegree", "Inner", "OuterCandidate",
232     "Digits", "Decomposable", "Obstruction", "Residual"};
233   If[!(And @@ (KeyExistsQ[t, #] & /@ keys)), Return[False]];
234   If[t["Type"] != "DegreeTest", Return[False]];
235   n = Length[c] - 1; d = t["RightDegree"];
236   If[!MemberQ[properDegrees[n], d], Return[False]];
237   m = Quotient[n, d]; If[t["OuterDegree"] != m, Return[False]];
238   If[!ListQ[t["Digits"]] || Length[t["Digits"]] != m + 1,
239     Return[False]];
240   h = prepare[t["Inner"], x]; f = prepare[t["OuterCandidate"], x];
241   digits = prepare[#, x] & /@ t["Digits"];
242   If[Length[h] != d + 1 || First[h] != 0 || Last[h] != 1,
243     Return[False]];
244   If[!(And @@ (Length[#] <= d & /@ digits)), Return[False]];
245   hm = Nest[multiply[#, h] &, {1}, m];
246   If[!(And @@ Table[
247     red[c[[n - k + 1]]/Last[c] - hm[[n - k + 1]]] === 0,
248     {k, 1, d - 1}]), Return[False]];
249   If[!zeroQ[subtract[c, digitCompose[digits, h]]], Return[False]];
250   If[!zeroQ[subtract[f, trim[First /@ digits]]], Return[False]];
251   obs = obstruction[digits];
252   If[t["Decomposable"] != (obs === None), Return[False]];
253   If[obs === None,
254     If[t["Obstruction"] != None, Return[False]],
255     If[!AssociationQ[t["Obstruction"]], Return[False]];
256     If[Lookup[t["Obstruction"], "DigitIndex", -1] != obs["DigitIndex"] ||
257       Lookup[t["Obstruction"], "Power", -1] != obs["Power"], Return[False]];
258     If[red[Lookup[t["Obstruction"], "Coefficient", Indeterminate] -
259       obs["Coefficient"]] != 0, Return[False]];
260     res = prepare[t["Residual"], x];
261     zeroQ[subtract[res, subtract[c, compose[f, h]]]]
262 ];
263
264 verifyData[c_List, data_, x_] := Module[{ds, ts, good, status, keys},
265   If[!AssociationQ[data] || !KeyExistsQ[data, "Type"], Return[False]];
266   If[data["Type"] == "DegreeTest", Return[verifyTest[c, data, x]]];
267   If[data["Type"] != "AllDegreeTests", Return[False]];
268   keys = {"InputDegree", "TestedRightDegrees", "AcceptedRightDegrees",

```



```

269   "Indecomposable", "Tests");
270   If[!(And @@ (KeyExistsQ[data, #] & /@ keys)), Return[False]];
271   ds = properDegrees[Length[c] - 1]; ts = data["Tests"];
272   If[data["InputDegree"] != vectorDegree[c] ||
273      data["TestedRightDegrees"] != ds || !ListQ[ts] ||
274      Length[ts] != Length[ds], Return[False]];
275   If[!(And @@ (verifyTest[c, #, x] & /@ ts)), Return[False]];
276   If[(#["RightDegree"] & /@ ts) != ds, Return[False]];
277   good = (#["RightDegree"] & /@ Select[ts, TrueQ[#["Decomposable"]]] &);
278   status = If[Length[c] < 3, Missing["NotApplicable", "DegreeBelowTwo"],
279             good === {}];
280   data["AcceptedRightDegrees"] === good && data["Indecomposable"] === status
281 ];
282
283 VerifyAlgebraicDecompositionData[p_, data_, x_Symbol] := Catch[
284   Module[{c, result}, c = prepare[p, x];
285     result = Quiet[Check[Catch[verifyData[c, data, x], $failureTag], False]];
286     TrueQ[result]], $failureTag];
287
288 AlgebraicDecompose[___] := invalid[];
289 AlgebraicDecompositions[___] := invalid[];
290 AlgebraicDecompositionPairs[___] := invalid[];
291 AlgebraicRightDecompose[___] := invalid[];
292 AlgebraicDecompositionData[___] := invalid[];
293 VerifyAlgebraicDecompositionData[___] := invalid[];
294 ComposeDecomposition[___] := invalid[];
295 VerifyAlgebraicDecomposition[___] := invalid[];
296
297 End[];
298 EndPackage[];

```